Events and the event handler

`Opened` and the remaining events are shown in the following listing. Like all events, they correspond to a command with the same root name.

Listing 12.9 Events

```
object Bet {
  ...

  sealed trait Event extends CborSerializable
  final case class MarketConfirmed(state: OpenState) extends Event
  final case class FundsGranted(state: OpenState) extends Event
  final case class ValidationsPassed(state: OpenState) extends Event
  final case class Opened(
      betId: String,
      walletId: String,
      marketId: String,
      odds: Double,
      stake: Int,
      result: Int)
      extends Event
  final case class Settled(betId: String) extends Event
  final case class Cancelled(betId: String, reason: String) extends Event
  final case class Failed(betId: String, reason: String) extends Event
  final case object Closed extends Event
  ...
}
```

Again, it is generally easier to understand the events and their relationship to the state than the relationships between states and commands. The event handler is generally easier to understand than the command handler. Each event comes from a command with the same root name. And the event's name pretty much tells you what state the entity will transition to.

Listing 12.10 Event handler

```
object Bet {
  ...
```